МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
имени М.В. ЛОМОНОСОВА

МЕХАНИКО-МАТЕМАТИЧЕСКИЙ ФАКУЛЬТЕТ

Отчёт о задаче №5

Алгоритм Брезенхема для рисования отрезков и окружностей.

Научный руководитель:
н.с. Романов Александр Вячеславович

Выполнила:
студентка 223 группы
Якшибаева Дарья Сергеевна

Москва
2025

Содержание

1 Описание задачи 3

2 Постановка задачи 3

3 Реализация задачи 4

4 Примеры выполнения 9

5 Вывод 10

1 Описание задачи

Задача состоит в написании программы на основе алгоритма Брезенхема и его аналога на языке Python.Результатом работы является графическое изображение с построенными фигурами в системе координат.
Алгоритм Брезенхема - алгоритм, определяющий, какие точки двумерного растра нужно закрасить, чтобы получить близкое приближение к прямой линии.

2 Постановка задачи

Входные данные:

- Параметры отрезка (координаты начала и конца)
- Параметры окружности (координаты центра и радиус)

Выходные данные:

Графический файл (output.png) с изображением:

- Отрезка, построенного по алгоритму Брезенхема
- Окружности, построенной по алгоритму Брезенхема

3 Реализация задачи

Основная идея алгоритма заключается в выборе пикселя, который находится ближе к идеальной математической линии, используя только целочисленные операции (сравнения и сложения).

Рассмотрим алгоритм Брезенхема для отрезка.
На каждом шаге выбор состоит из двух соседних пикселей:
(x+1,y) - горизонтальный шаг
(x+1,y±1) - диагональный шаг
Для упрощения алгоритма будем считать, что начальная точка (x_0,y_0) лежит левее по оси X, чем конечная (x_1,y_1).В противном случае, меняем их местами:

$$x_0,\, x_1 = x_1,\, x_0$$
$$y_0,\, y_1 = y_1,\, y_0$$

Для обеспечения плавности и точности рисования необходимо в начале проверить угол наклона прямой. Если угол наклона между прямой и осью X больше 45^0, то алгоритм меняет местами x и y , чтобы вести расчеты относительно оси Y, а не X.

$$\text{if dy>dx:}$$
$$x_0,\, y_0 = y_0,\, x_0$$
$$x_1,\, y_1 = y_1,\, x_1$$
$$\text{dx, dy} = \text{dy, dx ,}$$

где dy=(y_1-y_0) и dx=(x_1-x_0)
Нахождение "ошибки":
Введем вспомогательный параметр delta, измеряющий как текущее положение пикселя отклоняется от идеальной прямой.
Рассмотрим начальное значение delta и вычислим его.
Для удобства будем считать, что $y_0 < y_1$

$$F = \frac{(x - x_0)}{(x_1 - x_0)} - \frac{(y - y_0)}{(y_1 - y_0)}$$

$$F = y(x_1 - x_0) - x(y_1 - y_0) + C = ydx - xdy + C(1)$$

Точка M(x+1,y+$\frac{1}{2}$) является серединой между точками (x+1,y) и (x+1,y+1).Подст
ее в уравнение (1).

$$F = dy(x+1) - dx(y+\frac{1}{2}) + C$$

Можем домножить на два и убрать константу, так как данные операции не повлияют на знак выражения.

$$delta = 2dy(x+1) - dx(2y+1)$$

Для начальных условий:

$$delta = 2dy - dx$$

Если delta>0, то математическая линия располагается выше точки M по оси Y. Следовательно выбираем пиксель (x+1,y+1).
Если delta<0, то математическая линия располагается ниже точки M по оси Y. Следовательно выбираем пиксель (x+1,y).
После выбора между горизонтальным и диагональным шагом необходимо обновить ошибку.

$$\Delta delta = delta_{\text{new}} - delta = 2(F_{\text{new}} - F),$$

Горизонтальный шаг:
После горизонтального шага координата точки - (x+1,y)

$$\Delta delta = 2(dy(x+1) - dx*(y) - dy*(x) + dx*(y))$$

$$\Delta delta = 2dy$$

$$delta_{\text{new}} = delta + 2dy$$

Диагональный шаг:
После диагонального шага координата точки - (x+1,y+1)

$$\Delta delta = 2(dy(x+1) - dx(y+1) - dy * (x) + dx * (y))$$

$$\Delta delta = 2dy - 2dx$$

$$delta_{\text{new}} = delta + 2dy - 2dx$$

Данным алгоритмом необходимо пройти от x_0 до x_1.
for x in range(x0, x1 + 1):
if delta >= 0:
    delta -= 2 * dx
delta += 2 * dy

Рассмотрим аналог алгоритма Брезенхема для окружности.
Отличие алгоритма Брезенхема для окружности от алгоритма для прямой заключается в использовании симметрии, чтобы нарисовать только $\frac{1}{8}$ часть окружности (октант), а затем зеркально отражает эти точки для получения полной окружности.

$$points = [(x_0 + x, y_0 + y), (x_0 - x, y_0 + y),$$

$$(x_0 + x, y_0 - y), (x_0 - x, y_0 - y),$$

$$(x_0 + y, y_0 + x), (x_0 - y, y_0 + x),$$

$$(x_0 + y, y_0 - x), (x_0 - y, y_0 - x)]$$

Начальная точка:(0,r), где r - это радиус окружности.
Алгоритм работает в первом октанте (от 0^0 до 45^0), пока x не превысит y. После этого дальнейшие точки просто зеркально повторяются.

while x <= y:
plot_points(x, y)

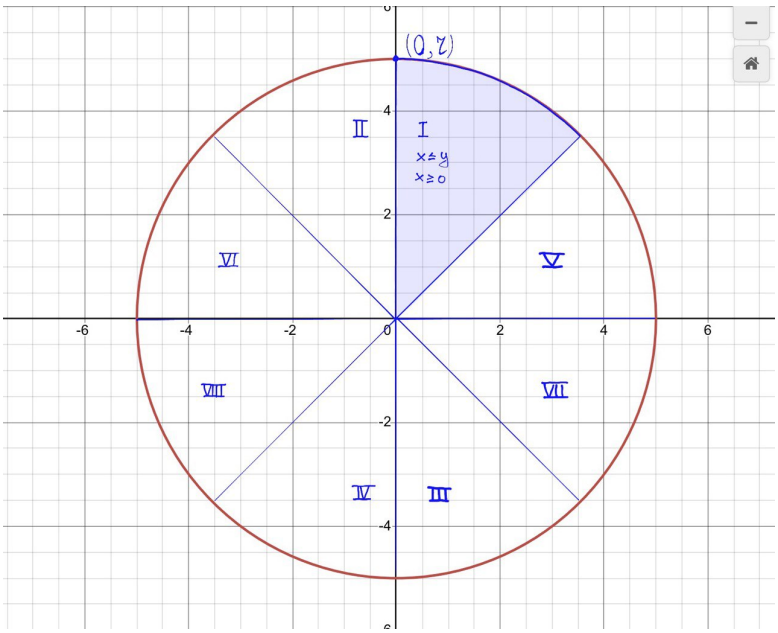

Рис. 1: Деление окр-ти на октанты

Аналогично случаю с прямой, введём вспомогательный параметр, который измеряет как текущее положение пикселя отклоняется от математической окружности.

$$F = x^2 + y^2 - r^2 (1)$$

Точка M(x+1,y-$\frac{1}{2}$) является серединой между точками (x+1,y) и (x+1,y-1).Подставим ее в уравнение (1) и применим начальные условия.

$$F(1, r - \frac{1}{2}) = 1^2 + (r - \frac{1}{2})^2 - r^2$$
$$F(1, r - \frac{1}{2}) = -r + \frac{5}{4}$$
$$delta = 5 - 4r$$

Разделим на два, так как данная операция не повлияет на знак параметра. В итоге:

$$delta = 3 - 2r$$

Если delta>0, то точка располагается вне окружности. Следовательно выбираем пиксель (x+1,y-1).
Если delta<0, то точка располагается внутри окружности. Следовательно выбираем пиксель (x+1,y).

После выбора между горизонтальным и диагональным шагом необходимо обновить ошибку.

$$\Delta delta = delta_{\text{new}} - delta = \frac{4(F_{\text{new}} - F)}{2},$$

Горизонтальный шаг:
После горизонтального шага (x+1,y) новая средняя точка - (x+2,y-$\frac{1}{2}$)

$$\Delta delta = 2((x+2)^2 + (y - \frac{1}{2})^2 - r^2 - (x+1)^2 - (y - \frac{1}{2})^2 + r^2)$$
$$\Delta delta = 2(2x+3)$$
$$delta_{\text{new}} = delta + 4x + 6$$

Диагональный шаг:
После диагонального шага (x+1,y+1) новая серединная точка - (x+2,y-$\frac{3}{2}$)

$$\Delta delta = 2((x+2)^2 + (y - \frac{3}{2})^2 - r^2 - (x+1)^2 - (y - \frac{1}{2})^2 + r^2)$$
$$\Delta delta = 2(2x - 2y + 5)$$
$$delta_{\text{new}} = delta + 4x - 4y + 10$$

Почему разный подход к перерасчету ошибки у окружности и прямой?
Прямая — это линейная функция, и ошибка меняется предсказуемо,поэтому достаточно сравнить два соседних пикселя.
Окружность — это квадратичная функция

$$(x^2 + y^2 = r^2),$$

и ошибка меняется нелинейно. Если просто сравнивать два шага, можно пропустить оптимальный пиксель.

Для обеспечения наглядности и корректности визуализации геометрических фигур в программе реализована декартова система координат.

4 Примеры выполнения

Начальные условия:

x_0: 0
y_0: 0
x_1: 0
y_1: 200

center X: 0
center Y: 0
radius: 200

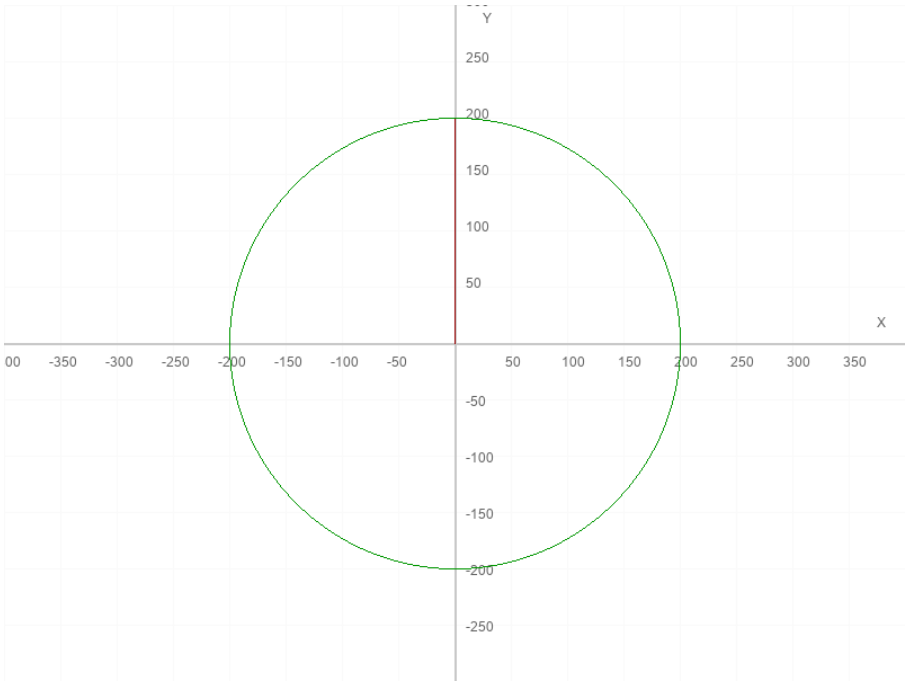

Рис. 2: Пример №1

x_0: 100
y_0: 10
x_1: -200
y_1: -100

center X: -100
center Y: 50
radius: 100

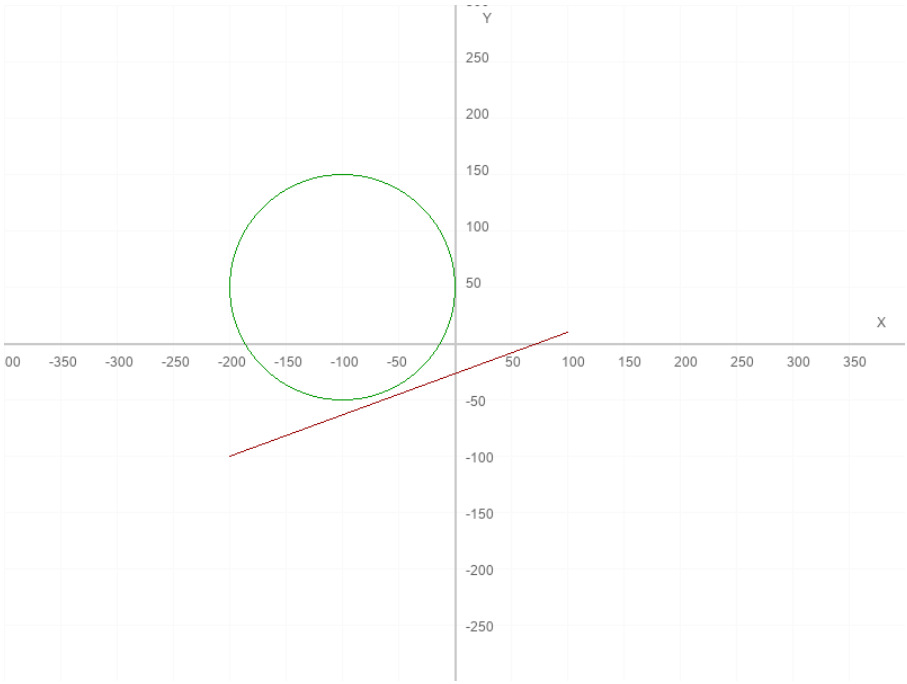

Рис. 3: Пример №2

5 Вывод

Реализованные алгоритмы Брезенхема успешно выполняют свою задачу по визуализации геометрических фигур, используя только целочисленные операции, а именно:
Для отрезков:

- Точно аппроксимирует прямые линии пикселями.
- Сохраняет "гладкость" при любых углах наклона.

Для окружностей:

- Обеспечивает 8-стороннюю симметрию.
- Сохраняет "гладкость" в любом октанте.

Инструменты проверки точности алгоритмов:

- Координатная сетка с шагом 50пикселей
- Цифровые подписи осей
- Контрастные цвета фигур